Structured Programming in Java

جامعة الكوفة / علوم الحاسوب والرياضيات

قسم : علوم الحاسوب

Lecture 9

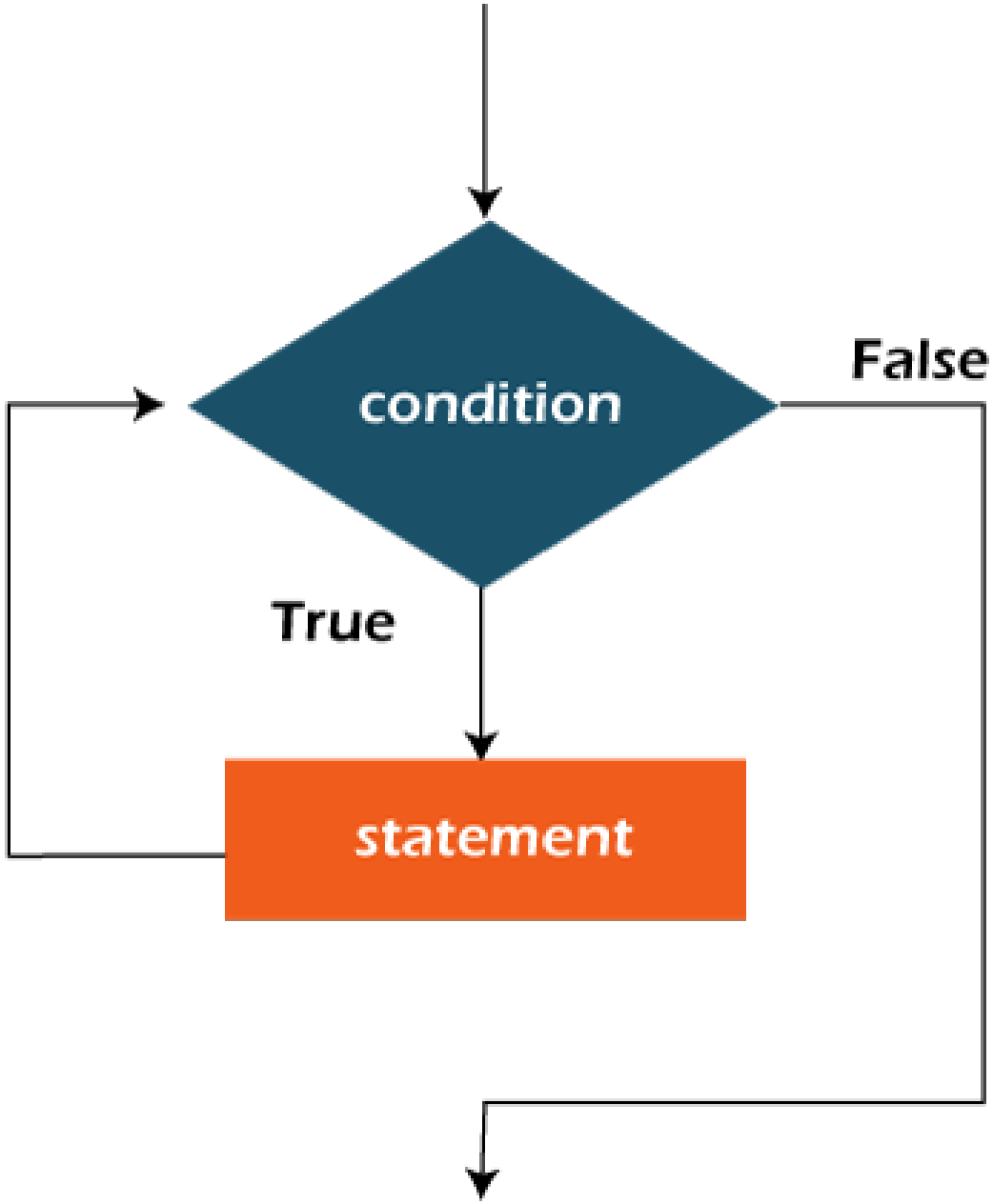
ا.م علي عبدالكريم حبيب

Java While Loop

- The Java while loop is used to iterate a part of the program repeatedly until the specified Boolean condition is true. As soon as the Boolean condition becomes false, the loop automatically stops.
- The while loop is considered as a repeating if statement. If the number of iteration is not fixed, it is recommended to use the while loop.

- **Syntax:**

```
while (condition) {
//code to be executed
I ncrement / decrement statement
}
```


condition
False
True
statement

Example of while statement:

```
public class WhileLoopExample1 {
public static void main(String[] args ) {
int counter=1;
while(counter<=10)
{
System.out.print(counter+" ");
++counter;
} } }
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> WhileLoopExample1 [Java Application] /Library/Java/JavaVirtualM
1 2 3 4 5 6 7 8 9 10
```

Example2: Write a Java program to find the factorial of 7 (i.e. 7! =1*2*3*4*5*6*7).

```
public class WhileLoopExample2 {
public static void main(String args[]){
int i=1,fact=1;
while(i<=7)
{
fact=fact*i;
++i;
} System.out.println("Factorial of 7 is: "+fact);  }  }
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> WhileLoopExample2 [Java Application] /Library/Java/JavaVirtualM
Factorial of 7 is: 5040
```

Example3: Write down a Java program to print the following shape:

1

2 2

3 3 3

4 4 4 4

5 5 5 5 5

6 6 6 6 6 6

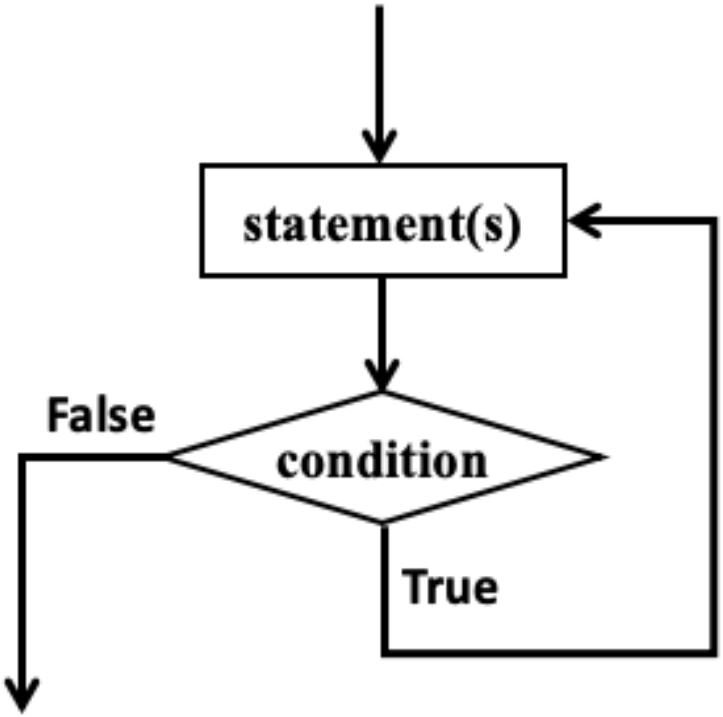
```
public class WhileLoopExample3 {
public static void main(String[] args) {
int i=1,j;
while (i <= 6)
{
j=1;
while (j <= i)
{
System.out.print(i+" ");
++j;
}
System.out.println();
++i;
}
 } }
```

Do-while Statement

It is similar to a while loop statement, except that the statement(s) in the loop is executed once even if the condition is false.

Syntax:

```
do {
statement(s);
} while(condition);
```



Example of do-while statement:

```
public class DoWhileLoopExample1 {
public static void main(String[] args ){
int c=1;
do {   System.out.print(c+" ");
++c;
}  while(c<=10);
} }
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> WhileLoopExample1 [Java Application] /Library/Java/JavaVirtualM
1 2 3 4 5 6 7 8 9 10
```

Example2: Write a Java program to draw the multiplication table of 8.

```
public class DoWhileLoopExample2 {
public static void main(String[] args) {
int i=1;
do {
System.out.println(8+" * "+i+" = "+8*i);
++i;
} while(i<=10);
}
}
```

Example3: Write a Java program to print the numbers divisible by 3 and 5 between 1 and 100.

```
public class DoWhileLoopExample3 {
public static void main(String[] args) {
int n=1;
do {
if ((n%3==0)&&(n%5==0))
System.out.print(n+" ");
++n;
} while(n<=100);
} }
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> DoWhileLoopExample3 [Java Application] /Library/Java/JavaVirtua
15 30 45 60 75 90
```

Homework: (Note: Solve these problems using **while** and **do-while)**

1) Write a Java program to enter five grades and find the average.

2) Write a Java program to find the outcome of the following equation:

X= 2! + 4! + 6! +……. 12!

3) White java code to enter three cases (1:count number of even numbers, 2: count odd numbers 3)

Use n=10, use <u>do- while</u> to repeat program until input i=0 exit from program.

Arrays:

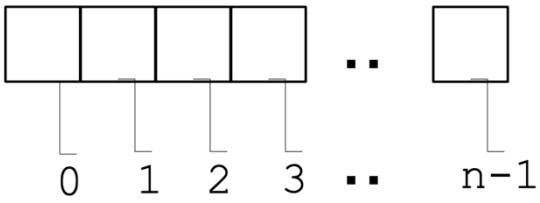
An array is an indexed list of values that all have the **same type**.

You can make an array of any type int, double, String, etc.

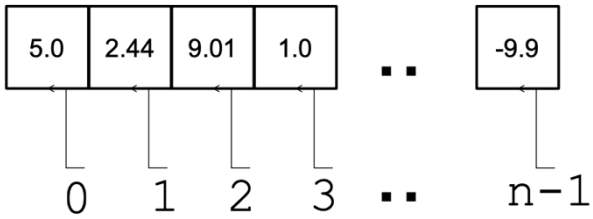
In array, the memory allocated is contiguous.

1. One-dimensional Array:

It is an array that has one index. **The index starts at zero and ends at length-1.**



Following an example of array of type double:



How to declare, initialize and access array?

An array is **defined** as following:

data_type array_name[];

OR

data_type[] array_name;

```
int[] values; // declare array of type int named “values”
int values[]; // same as above
```

To create an array of a given size, here size specifies the number of elements in the array, use the operator **new**:

```
double[] c = new double[5]; // declare an double array named “c” of size 5
double c[] = new double[5]; // same as above
```

Curly braces { } can be used to **initialize** an array. It can **ONLY** be used when you declare the variable.

```
int[] values = { 12, 24, -23, 47 }; // size of array deduced from the number of items
int values[] = { 12, 24, -23, 47 }; // same as above
```

Each array has a **length** variable built-in that contains the length of the array.

```
int[] values1 = new int[12]; // Declare and allocate a 12 elements int
array
int num = values1.length; // num is 12

int[] values2 = { 1, 2, 3, 4, 5 }
int size = values2.length; // size is 5
```

To access the elements of an array, use the [ ] operator. This operator contains the index value.

```
int[] values = { 12, 24, -23, 47 };
values[3] = 18; // {12, 24, -23, 18}
int x = values[1] + 3; // x = 27
```

Note: If the index is either outside the range of [0, length-1] or negative, Java Runtime will signal an exception called **ArrayIndexOutOfBoundException**.

Q> Is there an error in the following codes?

```
1. int[] values = {1, 2.5, 3, 3.5, 4};
2. char[] array = {'a', '*', '4', '/'};
   System.out.println(array[4]);
```

You can process all the elements of an array via a **loop.**

```
int[] values = { 12, 24, -23, 47 };
for (int i=0; i < values.length; i++)
{
System.out.println(values[i]);
}
```

```
12
24
-23
47
```

```
int[] values = { 12, 24, -23, 47 };
int i = 0;
while (i < values.length)
{
System.out.println(values[i]);
i++;
}
```

```
12
24
-23
47
```

```java
/* Java program to illustrate creating an array of type double and
   print each element with its index */
public class ArrayExample1 {
     public static void main(String[] args) {
     double[] array= {22.4, 13.3, -6.5, 9};
     for (int c = 0; c < array.length; c++)
     System.out.println("Element at index "+ c + " = " + array[c]);
     }
}
```

```
Problems | Javadoc | Declaration | Console | Progress
<terminated> ArrayExample1 [Java Application] /Library/Java/JavaVirtualMachin
Element at index 0 = 22.4
Element at index 1 = 13.3
Element at index 2 = -6.5
Element at index 3 = 9.0
```

Example: Write a Java program to find the summation of the elements in an array then print out the square root of that summation.

```java
public class ArrayExample2 {
     public static void main(String[] args) {
     int numbers[] = {8, 2, 6, 4, 3};
     int sum = 0;
     for (int i=0; i < numbers.length; i++)
          sum = sum + numbers[i];
     System.out.println("The summation is: " + sum);
     System.out.println("The square root is: " + Math.sqrt(sum));
     }
 }
```

```
Problems | Javadoc | Declaration | Console | Progress
<terminated> ArrayExample2 [Java Application] /Library/Java/JavaVirtualMachin
The summation is: 23
The square root is: 4.795831523312719
```

```java
// Java Example Read and Print Array
import java.util.Scanner;
public class ArrayExample3 {
   public static void main(String[] args) {

    // Declare variables
      int size;
      int[] items;
      Scanner in = new Scanner(System.in);

    // Enter the number of items i.e. the size of the array
      System.out.print("Enter the number of items: ");
      size = in.nextInt();

      items = new int[size]; // Allocate the array

      // Read the items of the array
      System.out.print("Enter the value of all items (separated
by space): ");
      for (int i = 0; i < items.length; i++)
         items[i] = in.nextInt();

      in.close();

    // Print array contents
      System.out.print("The array contains: ");
      for (int i = 0; i < items.length; i++)
       System.out.print(items[i] +" ");
   }
}
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> ArrayExample3 [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_112.jdk/Contents
Enter the number of items: 5
Enter the value of all items (separated by space): 33 8 25 -4 7
The array contains: 33 8 25 -4 7
```

Lecture //Programs Examples on 1 D Array

أ.م.د علي عبدالكريم حبيب

جامعة الكوفة – كلية علوم الحاسوب والرياضيات – قسم علوم الحاسوب

Q1: Write Java program to create array have 5 string items {Ali, Hassan , Mohaamed, Jaafar}

Then search for word Ali , if his name in array , **print found if not print not found**.

```
public class find {
   /**
    * @param args the command line arguments
    */
   public static void main(String[] args) {
      // TODO code application logic here
       String names[]={"Ali","Hassah","Jaafar","Mohamed"};
      int x=0;// flag
      for(int i=0;i<names.length;++i)
      {    if(names[i]=="Ali")
           x=1;        }
      if (x==1) System.out.println("Found");
      else System.out.println("not Found");
   }   }
```

Q2: Write java codes to create array with any five integer items { 5,4,2,1} then sorting array in ascending order.

```
public class SortArr{
**/
param args the command line arguments@ *
```

```
/*
public static void main(String[] args){
TODO code application logic here //
;{5,4,2,1}=int []a

;int swap
for(int i=0;i<a.length;++i)
{
for(int j=i+1;j<a.length;++j)
if (a[i]>a[j])
{
;swap=a[i]
;a[i]=a[j]
;a[j]=swap
{
{
for print //
for(int i=0;i<a.length;++i)
;System.out.println(a[i])

}


}
```

Q3: Write java codes to create array with any five integer items {10,20,60,70,30} then find Max and Min Numbers in array

H.W//

1- Rewrite all above programs and input items from Keyboards

2- Rewrite second program and input char instead of string from keyboard

End

Lecture //Programs Examples on 1 D Array

أ.م.د علي عبدالكريم حبيب

جامعة الكوفة – كلية علوم الحاسوب والرياضيات – قسم علوم الحاسوب

Q1: Write Java program to create array have 5 string items {Ali, Hassan , Mohaamed, Jaafar}

Then search for word Ali , if his name in array , **print found if not print not found**.

```
public class find {
   /**
    * @param args the command line arguments
    */
   public static void main(String[] args) {
      // TODO code application logic here
       String names[]={"Ali","Hassah","Jaafar","Mohamed"};
      int x=0;// flag
      for(int i=0;i<names.length;++i)
      {    if(names[i]=="Ali")
           x=1;        }
      if (x==1) System.out.println("Found");
      else System.out.println("not Found");
   }   }
```

Q2: Write java codes to create array with any five integer items { 5,4,2,1} then sorting array in ascending order.

```
public class SortArr{
**/
param args the command line arguments@ *
```

```
/*
public static void main(String[] args){
TODO code application logic here //
;{٥,٤,٢,١}=int []a

;int swap
for(int i=0;i<a.length;++i)
{
for(int j=i+1;j<a.length;++j)
if (a[i]>a[j])
{
;swap=a[i]
;a[i]=a[j]
;a[j]=swap
{
{
for print //
for(int i=0;i<a.length;++i)
;System.out.println(a[i])

}


}
```

Q3: Write java codes to create array with any five integer items {10,20,60,70,30} then find Max and Min Numbers in array

H.W//

1- Rewrite all above programs and input items from Keyboards

2- Rewrite second program and input char instead of string from keyboard

End

Example 3: Consider the following 2D array named temperatures:

	0	1	2	3	4	5	6
0	12.5	12.2	14.0	13.0	13.2	13.0	12.2
1	**15.2**	**11.9**	**11.8**	**13.2**	**12.7**	**14.5**	**12.4**
2	9.75	11.8	11.9	13.5	12.7	14.7	13.0
3	14.0	13.2	11.8	14.0	14.3	11.0	11.9

Perform the following:

1. Print out the value of highlighted indexes.
2. Print out the value of highlighted row.
3. Find out the maximum, minimum and average values.

```
public class Array2DExample3 {
     public static void main(String args[]) {
     double[][] temperatures = {{12.5,12.2,14.0,13.0,13.2,13.0,12.2},
                                {15.2,11.9,11.8,13.2,12.7,14.5,12.4},
                                {9.75,11.8,11.9,13.5,12.7,14.7,13.0},
                                {14.0,13.2,11.8,14.0,14.3,11.0,11.9}};
     double min,max,sum;
     min= temperatures[0][0]; // Assume the first item is the minimum
     max= temperatures[0][0]; // Assume the first item is the maximum
     sum=0;
     System.out.println(temperatures[1][5]);

     for (int col=0; col< temperatures[1].length; col++)
          System.out.print(temperatures[1][col] + " ");
     System.out.println();

     for (int row=0; row< temperatures.length; row++)
     {
        for (int col=0; col< temperatures[row].length; col++)
          {
            if (min > temperatures[row][col])
                    min= temperatures[row][col];
            if (max < temperatures[row][col])
                    max= temperatures[row][col];
               sum=sum+ temperatures[row][col];
          }
     }
     System.out.println("The minimum value is " + min);
     System.out.println("The maximum value is " + max);
     System.out.println("The average is " + sum/28);
   }
}
```

```
Markers   Properties   Servers   Data Source
<terminated> Array2DExample3 [Java Application] /Library
14.5
15.2 11.9 11.8 13.2 12.7 14.5 12.4
The minimum value is 9.75
The maximum value is 15.2
The average is 12.833928571428572
```

Example 4: Write down a Java program to read a 2-dimensional array x[4][4] then perform each of the following:

1. Find the maximum and minimum in the third row
2. Replace the two diagonals with 1

```
import java.util.Scanner;
public class Array2DExample4 {
     public static void main(String args[]) {
    int [][] x = new int[4][4];
    int min,max;
    Scanner s = new Scanner(System.in);
    System.out.print("Enter the array elements: ");
    for (int i=0; i< 4; i++)
          for (int j=0; j< 4; j++)
          x[i][j] = s.nextInt();
    s.close();

    for (int i=0; i< 4; i++)
    {
        for (int j=0; j< 4; j++)
                System.out.print(x[i][j] +" ");
     System.out.println();
     }
    min= x[3][0]; // Assume the first item is the minimum
    max= x[3][0]; // Assume the first item is the maximum

    for (int j=0; j< 4; j++)
    {
       if (min > x[3][j])
          min= x[3][j];
       if (max < x[3][j])
          max= x[3][j];
     }
     System.out.println("The minimum value is " + min);
     System.out.println("The maximum value is " + max);

     for (int i=0; i< 4; i++)
     for (int j=0; j< 4; j++)
     {
                if (i == j) // Condition for principal diagonal
                     x[i][j] = 1;
                if ((i+j) == 3)// Condition for secondary diagonal
                     x[i][j] = 1;
     }
     for (int i=0; i< 4; i++)
     {
     for (int j=0; j< 4; j++)
                System.out.print(x[i][j] +" ");
     System.out.println();
     }
  }
}
```

```
Markers  Properties  Servers  Data Source Explorer  Snippets  Console
<terminated> Array2DExample4 [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_112.jdk/
Enter the array elements: 22 12 23 24 44 13 32 34 55 18 17 90 12 14 78 55
22 12 23 24
44 13 32 34
55 18 17 90
12 14 78 55
The minimum value is 12
The maximum value is 78
1 12 23 1
44 1 1 34
55 1 1 90
1 14 78 1
```

Homework:

Q1> Write a Java program to count the number of occurrences of an element in an 2D array.

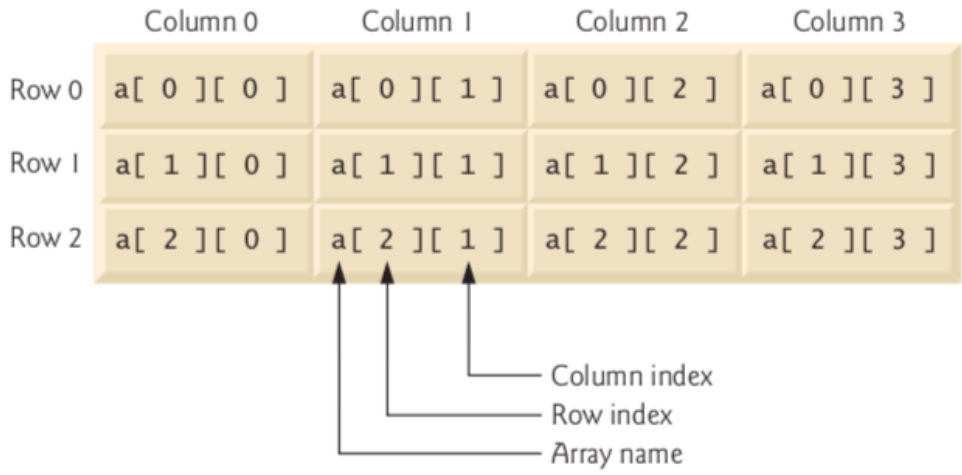
Q2> Write a Java program to print negative numbers in an 2D array.

Q3> Write a Java program to read of an 2D array then to print the two diagonals elements only.

2. Two-dimensional Array:

Arrays that require two indexes to identify a particular element.

The following figure shows two-dimensional (2D) array named a contains 3 rows and 4 columns:



Note:

- The indexes of the array start with *zero* always.
- The lengths of the rows in 2D array are *not* required to be the same.
- Array could extend naturally to 3D, 4D, and even higher-dimensional, but they are not used very often in practice.

How to declare, initialize and access 2D array?

The syntax for creating 2D Array as following:

data_type array_name[][];

OR

data_type[][] array_name;

```
int[][] array; // Declare 2D array of type int named “array”
int array[][]; // Same as above
```

Specifying a 2D array dimensions:

```
double[][] c = new double[4][5]; // Declare an double 2D array named
“c” with 4 rows and 5 columns
double c[][] = new double[4][5]; // Same as above
```

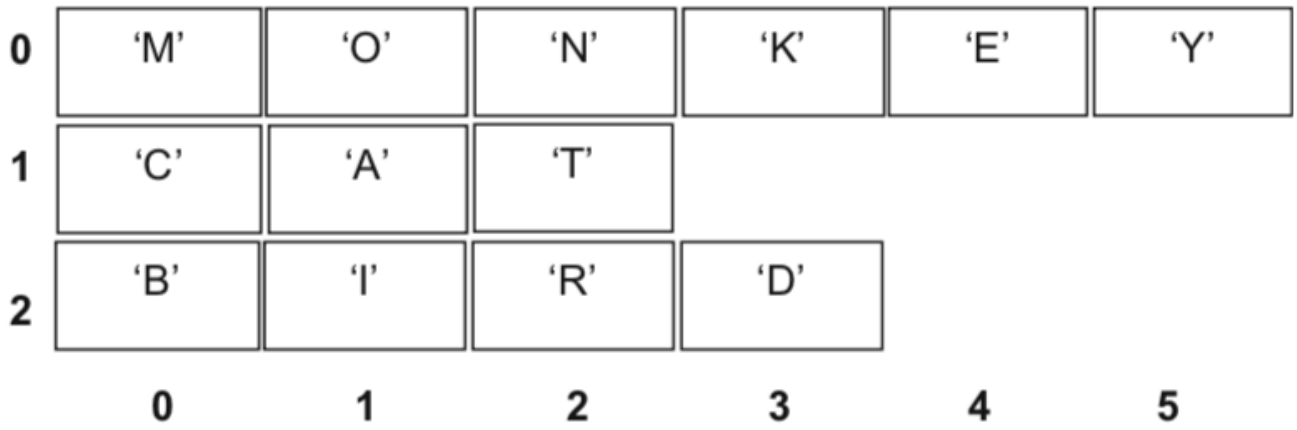
Like one-dimensional arrays, 2D arrays could be declare and initialize:

```
int[][]  A  =  {
                 {11, -25, 4, 77},
                 {-21, 55, 43, 11}
               };// This creates and initializes a 2D array of int
with 2 rows and 4 columns

int[][] A = new int[2][4]; // Same as above
// Initialize first row of values
A[0][0] = 11;
A[0][1] = -25;
A[0][2] = 4;
A[0][3] = 77;
// Initialize second row of values
A[1][0] = -21;
A[1][1] = 55;
A[1][2] = 43;
A[1][3] = 11;
```

A two-dimensional array with a variable number of columns is called a *ragged array*. For example:

```
char[][] animals = {
                        {'M', 'O', 'N', 'K', 'E', 'Y'}, // 6 columns
                        {'C', 'A', 'T'}, // 3 columns
                        {'B', 'I', 'R', 'D'} // 4 columns
                    }; // this creates a 2D array with a variable
number of columns
```



We could use the *length attribute* to gets array information:

```
int array[][] = new int[12][8];   // a 12×8 array of int
System.out.println(array.length);      // 12 number of rows
System.out.println(array[0].length);   // 8 number of columns
System.out.println(array[11].length);  // 8 number of columns
```

To access an element in a 2D array requires both a row and a column index. And a *pair* of nested loops is usually used to process each element.

```
public class Array2DExample1 {
      public static void main(String args[]) {
      // Declaring and initializing 2D array
      int arr[][] = { {1,2,3},{4,5,6},{7,8,9} };
      // Printing 2D array
      for (int i=0; i< 3; i++)
            {
            for (int j=0; j< 3; j++)
                  {
                  System.out.print(arr[i][j] + " ");
                  }
            }
      }
}
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> Array2DExample1 [Java Application] /Library/Java/JavaVirtualMac
1 2 3 4 5 6 7 8 9
```

```
public class Array2DExample2 {
      public static void main(String args[]) {
       // Declaring and initializing ragged array
       char[][] animals= {
                           {'M', 'O', 'N', 'K', 'E', 'Y'},
                           {'C', 'A', 'T'},
                           {'B', 'I', 'R', 'D'}
                         };
       for (int row = 0; row < animals.length; row++)
            {
            for (int col = 0; col < animals[row].length; col++)
                  {
                  System.out.print(animals[row][col]); // Display one character
                  }
                 System.out.println(); // New line after one row display
             }
       }
}
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> Array2DExample2 [Java Application] /Library/Java/JavaVirtualMac
MONKEY
CAT
BIRD
```

Java Methods

A **method** is a block of code which only runs when it is called. Methods are used to perform certain actions, and they are also known as **functions**.

So, the method used to reuse code: define the code once, and use it many times.

Create a Method

A method must be declared within a class. It is defined with the name of the method, followed by parentheses **()**. Java provides some pre-defined methods, such as `System.out.println()`, but you can also create your own methods to perform certain actions:

```
returnType methodName() {

  // method body

}
```

- returnType - It specifies what type of value a method returns For example if a method has an int return type then it returns an integer value.

  If the method does not return a value, its return type is **void**.
- methodName - It is an identifier that is used to refer to the particular method in a program.
- method body - It includes the programming statements that are used to perform some tasks. The method body is enclosed inside the curly braces { }

Method With No Return Value No pass parameter

This type of method has no pass parameter no return value

Example:

```
static void myMethod()
```

`Static`: means that the method belongs to the class and not an object of the class.

`Void`: means that this method does not have a return value.

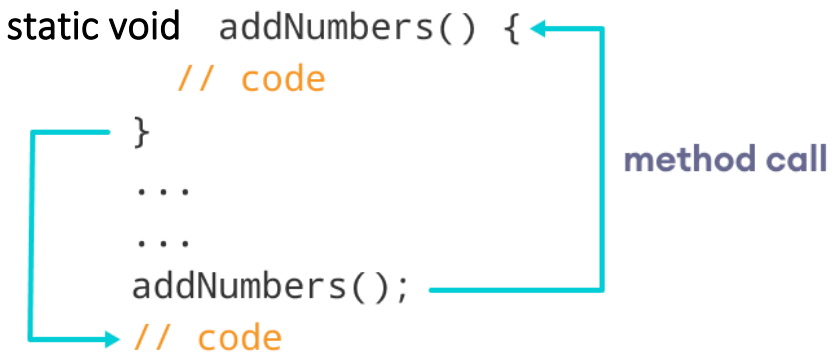
`myMethod()`: is the name of the method.

Call a Method

To call a method in Java, write the method's name followed by two parentheses **()** and a semicolon**;**

In the following example, `addNumbers()` is used to print a sum of two numbers (the action), when it is called:

```
// calls the method
addNumbers();
```



Example 1:

```
public class calcuMethod {
   static int num1=25;
   static int num2=2;
   //create method
    static void addNumbers(){
      int sum=num1+num2;
         System.out.println("sum="+sum);
      }
public static void main (String arg[]){
 //call method
 addNumbers();
}   }
```

****

Example 2:

```
public class calcuMethod {
   static int num1=25;
   static int num2=2;
   //create method
    static void addNumbers(){
      int sum=num1+num2;
      System.out.println("sum="+sum);
      }
 static void multiNumbers(){
      int mul=num1*num2;
         System.out.println("mul="+mul);
      }
```

```
public static void main (String arg[]){
 //call method
 addNumbers();
 multiNumbers();
}
  }
```

2- Method Pass Parameter no Return value

Information can be passed to methods as parameter. Parameters act as variables inside the method.

Parameters are specified after the method name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma.

The following example has a method that takes a String called fname as parameter. When the method is called, we pass along a first name, which is used inside the method to print the full name:

Example

```
public class Main {
  static void myMethod(String fname) {
    System.out.println(fname + " Refsnes");
  }
public static void main(String[] args) {
    myMethod("Liam");
    myMethod("Jenny");
    myMethod("Anja");
  }
}
Output:
// Liam Refsnes
// Jenny Refsnes
// Anja Refsnes
```

Multiple Parameters

```
public class Main {
  static void myMethod(String fname, int age) {
    System.out.println(fname + " is " + age);
  }
  public static void main(String[] args) {
    myMethod("Liam", 5);
    myMethod("Jenny", 8);
    myMethod("Anja", 31);
  }
}
Output:
// Liam is 5
// Jenny is 8
// Anja is 31
```

Example 3 : java code to pass three degree **avstudent**(St, d1,d2,d3) for three students then find average

Class **stud**{

}(static void **avstudent**(string st, int d1, int d2, int d3

; int sum=d1+d2+d3

; double av=sum/3

; (System.out.println(st+": "+" "+av="+av

{

```
Public static void main( String args[]){

avstudent("Ali",10, 10, 10);

avstudent("Hussian",20, 10, 20);

avstudent("Mohammed",50, 100, 50);

 }
```

Output :

Ali : av=10

Hussian: av=16.6

Mohammed: av=66.666

H.W:

1- write java program to create three methods area of circle, circumference of a circle, and area of area of the triangle, pass parameter not return value.

2- write code of java to sort x1[5], x2[4] using SortArr().

Method With Return Value and pass parameter

This type of method has pass parameter and return value
Example:

In the following example, `addNumbers(int a, int b)` is used to print a sum of two numbers (the action), when it is called:

Here, we are returning the variable `sum`. Since the return type of the function is `int`. The sum variable should be of `int` type. Otherwise, it will generate an error.

Example 1: find sum of two numbers

```
public class calcuMethod {
   //create method
    static int addNumbers(int num1,int num2){
      int Add=num1+num2;
        return Add;
      }
public static void main (String arg[]){
 //call method
int sum=0;
int a=25;
int b=2;
sum=addNumbers(a, b);
}   }
```

****

Example 2: find sum and multiplication of two numbers

```
public class calcuMethod {
   //create method
    static float AvNumbers(int a, int b){
      int Av=(a+b)/2;
```

```
        retrun Av;
    }
 static int multiNumbers(int num1, int num2){
    int mul=num1*num2;
        return mul;
    }


public static void main (String arg[]){
 //call method
 float av=AvNumbers(5,2);
 int mul=multiNumbers(5,5);
}
  }
```

Example 3) program to pass number and find factorial using methos **int fact(a)**

```
Class factNum{

Static int fact(int a){
int f=1;
For(int i=1;i<a;++i)
f=f*i;
return f;
}// end method
public static void main (String arg[]){
int a=5;
int ff=fact(a);
Sysyem.out.println("factoial"+a+" ="ff); } // end main }// end class
```

Example 4: find areaTr and areaCi

```
public class calcuArea{
   //create method
    static float areaT(int b, int h){
      float ar=(b+h)/2;
         retrun ar;
      }
 static int areaCi(int r){
      int arCi=3.14*(r*r);
         return arCi;
      }


public static void main (String arg[]){
 //call method
 float arT= areaT(5,2);
 float arCi= areaCi(4);
}
  }
```

H.W:- Write java program to pass array x[5] as integer values , then store items of array from index 2 to end of array in new array xnew[]; then return xnew to main method.

Java String

أ.م.د علي عبدالكريم حبيب الرماحي

What is String in Java?

- Generally, String is a sequence of characters. But in Java, string is an object that represents a sequence of characters. The java.lang.String class is used to create a string object.

How to create a string object?

- There are two ways to create String object:

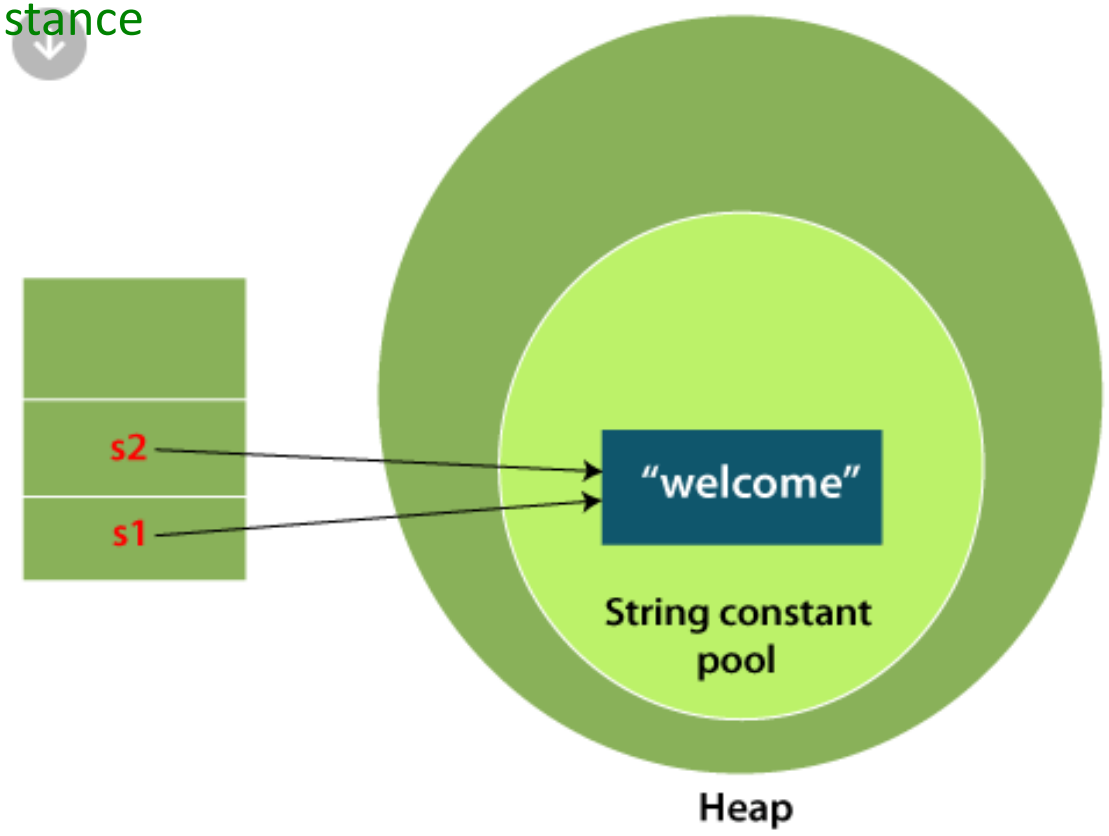
1. By string literal
2. By new keyword

String Literal

- Java String literal is created by using double quotes. For Example:
- String s="welcome";
- Each time you create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool. For example:

String Literal … Cont

1. String s1="Welcome";
2. String s2="Welcome";//It doesn't create a new instance



- In the above example, only one object will be created. Firstly, JVM will not find any string object with the value "Welcome" in string constant pool that is why it will create a new object. After that it will find the string with the value "Welcome" in the pool, it will not create a new object but will return the reference to the same instance.

By new keyword

```
String s=new String("Welcome");//creates two objects and one referen
ce variable
```

In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the string constant pool. The variable s will refer to the object in a heap (non-pool).

Java String Example

```
public class StringExample{
public static void main(String args[]){
String s1="java";//creating string by Java string literal
char ch[]={'s','t','r','i','n','g','s'};
String s2=new String(ch);//converting char array to string
String s3=new String("example");//creating Java string by new keyword

System.out.println(s1);
System.out.println(s2);
System.out.println(s3);
 }}
```

Java String class methods

- Java String charAt()

```
char Variable =charAt(int index)
```

Example :

FileName: CharAtExample.java

```
public class CharAtExample{
public static void main(String args[]){
String name="javatpoint";
char ch=name.charAt(4);//returns the char value at the 4th index
System.out.println(ch);
}}
```

- Java String length()

Return length of string

- String.length()

Ex:FileName: LengthExample.java

```
public class LengthExample{
public static void main(String args[]){
String s1="javatpoint";
String s2="python";
System.out.println("string length is: "+s1.length());//10 is the length of ja
vatpoint string
System.out.println("string length is: "+s2.length());//6 is the length of pyt
hon string
}}
```

- Java String substring() method example

```
public class SubstringExample{
public static void main(String args[]){
String s1="javatpoint";
System.out.println(s1.substring(2,4));//returns va// from 2 to 4-1 index
System.out.println(s1.substring(2));//returns vatpoint // from 2 to end
}}
```

Java String contains()

- The **Java String class contains()** method searches the sequence of characters in this string. It returns *true* if the sequence of char values is found in this string otherwise returns *false*.

- **FileName:** ContainsExample.java

```
class ContainsExample{
public static void main(String args[]){
String name="what do you know about me";
System.out.println(name.contains("do you know"));
System.out.println(name.contains("about"));
System.out.println(name.contains("hello"));
}}
```

Java String equals()

- The **Java String class equals()** method compares the two given strings based on the content of the string. If any character is not matched, it returns false. If all characters are matched, it returns true.

  The String equals() method overrides the equals() method of the Object class.

- **FileName:** EqualsExample.java

```
public class EqualsExample{
public static void main(String args[]){
String s1="javatpoint";
String s2="javatpoint";
String s3="JAVATPOINT";
String s4="python";
System.out.println(s1.equals(s2));//true because content and case is same
System.out.println(s1.equals(s3));//false because case is not same
System.out.println(s1.equals(s4));//false because content is not same
}}
```

Java String isEmpty()

```
public class IsEmptyExample{
public static void main(String args[]){
String s1="";
String s2="javatpoint";

System.out.println(s1.isEmpty());
System.out.println(s2.isEmpty());
}}
```

Java String split()

- The **java string split()** method splits this string against given regular expression and returns a char array.

Ex

```
public class Split Example{
public static void main(String args[]){
String s1="java string split method by javatpoint";
String[] words=s1.split("\\s");//splits the string based on whitespace
//using java foreach loop to print elements of string array
for(String w:words){
System.out.println(w);
}
}}
```

Java String toLowerCase()

- The **java string toLowerCase()** method returns the string in lowercase letter. In other words, it converts all characters of the string into lower case letter.
- Java String toLowerCase() method example

```
public class StringLowerExample{
public static void main(String args[]){
String s1="JAVATPOINT HELLO stRIng";
String s1lower=s1.toLowerCase();
System.out.println(s1lower);
}}
```

- Java String toUpperCase()

The **java string toUpperCase()** method returns the string in uppercase letter. In other words, it converts all characters of the string into upper case letter.

- Java String toUpperCase() method example

```
public class StringUpperExample{
public static void main(String args[]){
String s1="hello string";
String s1upper=s1.toUpperCase();
System.out.println(s1upper);
}}
```

Trim Function

The **Java String class trim()** method eliminates leading and trailing spaces. The Unicode value of space character is '\u0020'. The trim() method in Java string checks this Unicode value before and after the string, if it exists then the method removes the spaces and returns the omitted string.

```
public class StringTrimExample2 {
   public static void main(String[] args) {
      String s1 ="  hello java string   ";
      System.out.println(s1.length());
      System.out.println(s1); //Without trim()
      String tr = s1.trim();
      System.out.println(tr.length());
      System.out.println(tr); //With trim()
   }
}
```